AI Maxx IDE

A self-hosted remote development companion for Cursor IDE.

The problem with remote development is that most solutions solve it by handing over the entire machine — RDP, VNC, full SSH access. That works, but it exposes everything, runs heavy on both ends, and is genuinely painful on a mobile screen.

AI Maxx IDE takes a different approach. Instead of exposing the machine, it exposes specific capabilities: browse configured projects, search the codebase, run AI-assisted workflows, execute terminal commands, and when absolutely necessary, take direct control of the screen. Each capability is scoped, intentional, and mobile-first in its interaction design.

The backend is a Django ASGI server packaged as a Windows executable. The frontend is a Flutter mobile app. Connectivity is handled by Cloudflare Tunnel — no firewall configuration, no exposed ports, no VPN. The index covers 26,720 files, built on startup and kept in memory for instant search.

Frontend	Flutter — Android & iOS
Backend	Django ASGI — async WebSocket + HTTP
AI	Plan Mode (analysis) + Agent Mode (execution), configurable model
Search	In-memory full-workspace index — 26,720 files, synced on start
Terminal	Subprocess PTY per tab, streamed over WebSocket
Remote	Screen stream + mouse/keyboard injection via Remote tab
Tunnel	Cloudflare Tunnel — internet exposure without open ports
Packaging	Windows EXE + batch launcher, no Python install required

Project Explorer & Workspace Index

The first design decision in AI Maxx IDE was to reject the idea of full remote access. Most remote development tools — RDP, VNC, SSH tunnels — hand over the entire machine. That's both a security risk and cognitive overload on a small screen.

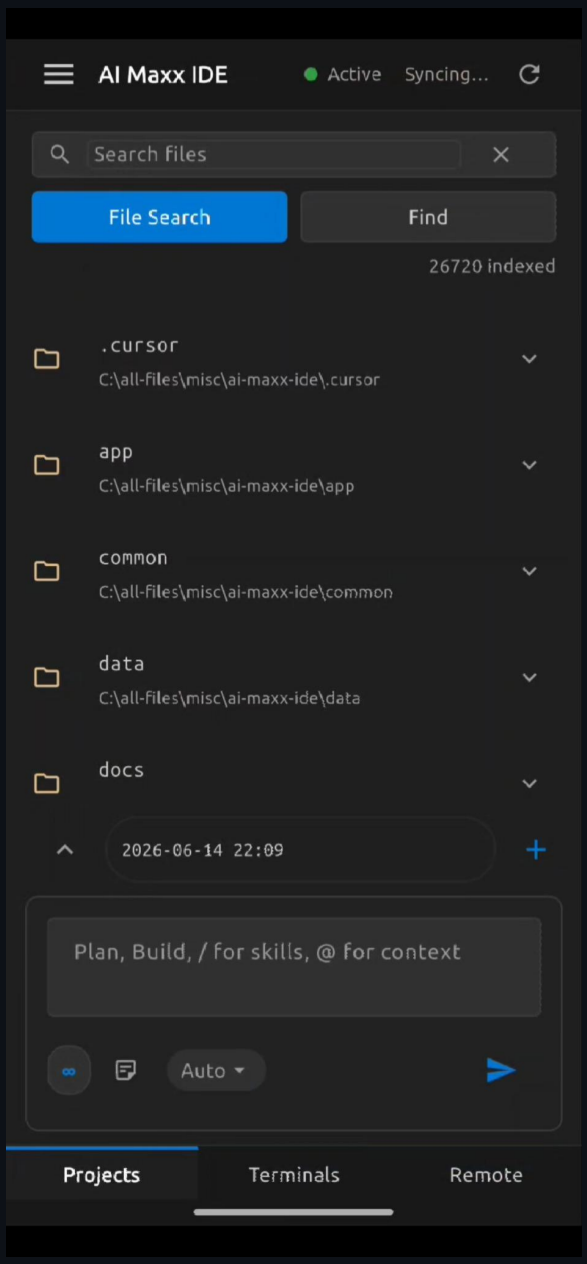
Instead, the project explorer only surfaces what's been explicitly configured. Each folder in this list represents a project root the developer chose to expose. Nothing outside these paths is reachable through the app.

The 26,720 file index is built on startup and kept in memory. This makes search instant — no file system traversal at query time. The session panel at the bottom shows the last active conversation timestamp, reinforcing that context persists between connections. You pick up exactly where you left off.

Problem solved
Full-machine exposure replaced with scoped project access

Index strategy
In-memory index built on startup for zero-latency search

Session model
Conversations are persistent — resumable across devices



Code Search — File & Grep

Navigating a large codebase on mobile is where most remote tools fall apart. Scrolling a file tree to find a file across 26,000 entries is unusable.

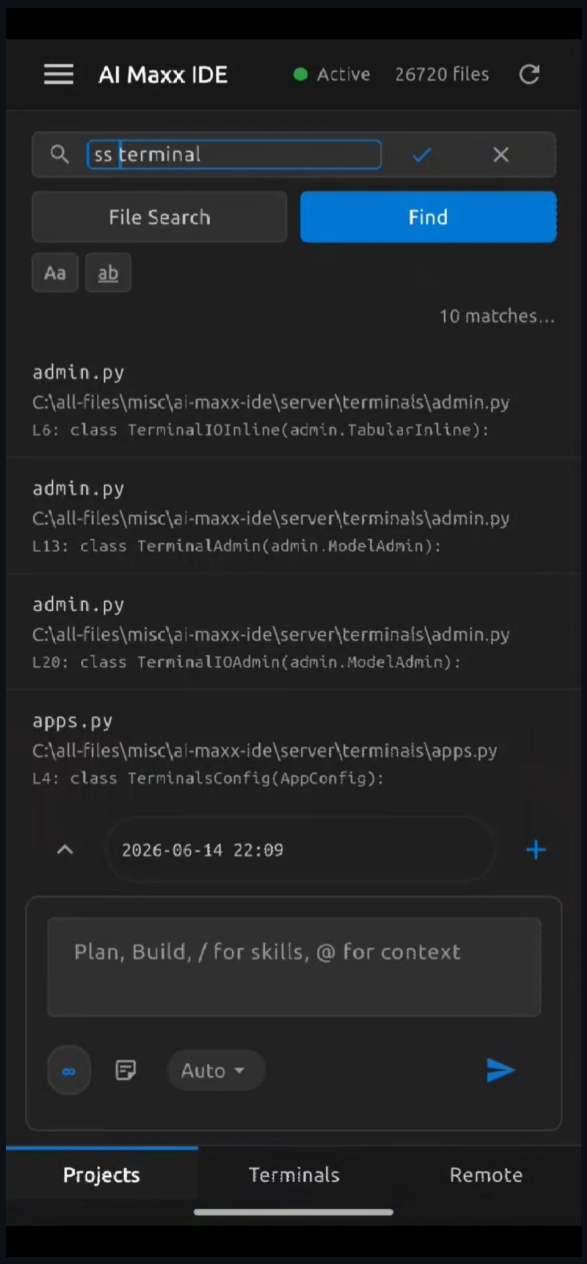
The search panel solves this with two modes. File Search is a fuzzy filename matcher — the equivalent of Ctrl+P in VS Code or Cursor. Type a fragment, get ranked results instantly. Find mode runs a grep across file contents, returning every match with its file path and line number shown inline.

The distinction matters: you use File Search when you know what file you want, and Find when you know what the code looks like but not where it lives. Both are built for the mental model of a developer who already knows the codebase, just needs to navigate it quickly from a phone.

File Search
Fuzzy filename match — Ctrl+P equivalent, instant results

Find / Grep
Full-text search across all indexed files with line-level output

Use case
Navigation tool for developers who know the codebase, not explorers



AI Context Injection via @path

The AI assistant's most important design choice is how context is provided. Rather than sending the entire codebase to the model — expensive, slow, and often irrelevant — the developer explicitly attaches what matters using the @ prefix.

Typing @path resolves to a file or folder from the indexed workspace. The assistant receives that content as context alongside the prompt. This keeps token usage lean and forces intentionality: you decide what the AI sees.

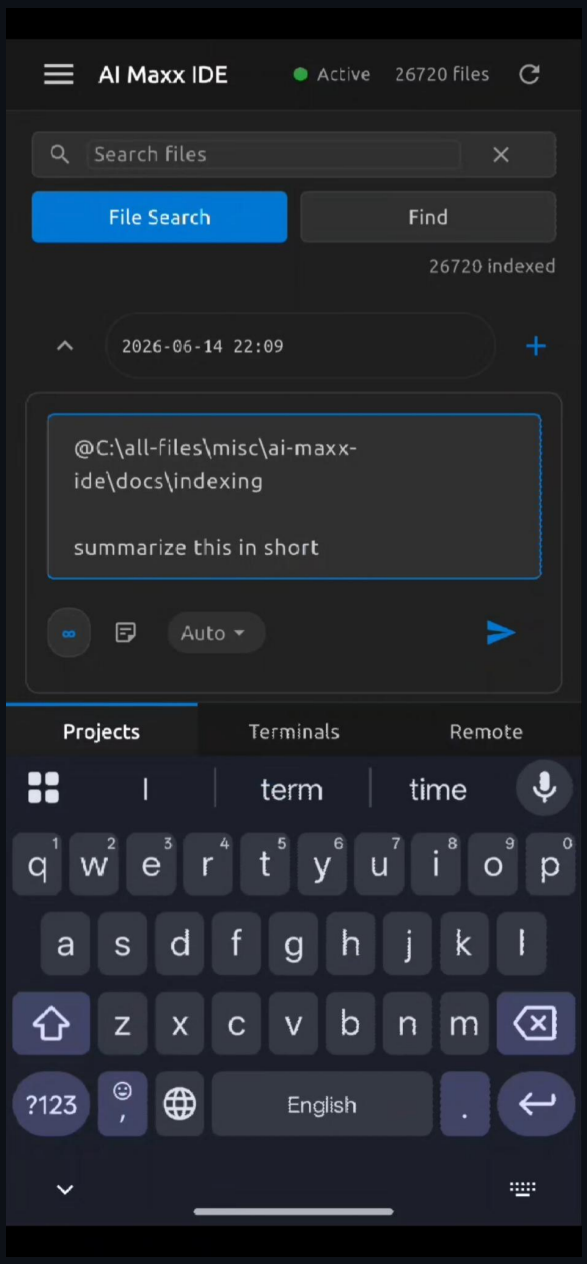
The / prefix loads skills — pre-written instruction sets for common tasks like code review, documentation, or refactoring. This lets teams standardise how the AI approaches recurring work without re-explaining it every session.

The model selector is always visible. Switching between a fast model for quick lookups and a capable model for deep planning is a first-class workflow, not buried in settings.

@path context
Explicit file/folder attachment — developer controls what the AI sees

/ skills
Pre-written instruction sets for standardised recurring tasks

Model selector
Visible, switchable — fast vs capable as a first-class choice



AI Response — Structured Output

The response view is where the value of the context injection becomes visible. The assistant doesn't just return raw text — it renders structured markdown inline: headings, bold, sections, and hierarchical content that mirrors how a developer would write documentation.

In this example, attaching the docs/indexing folder and asking for a summary produced a properly structured document — title, section headers, and categorised content. This is the Plan Mode behaviour: read, analyse, explain.

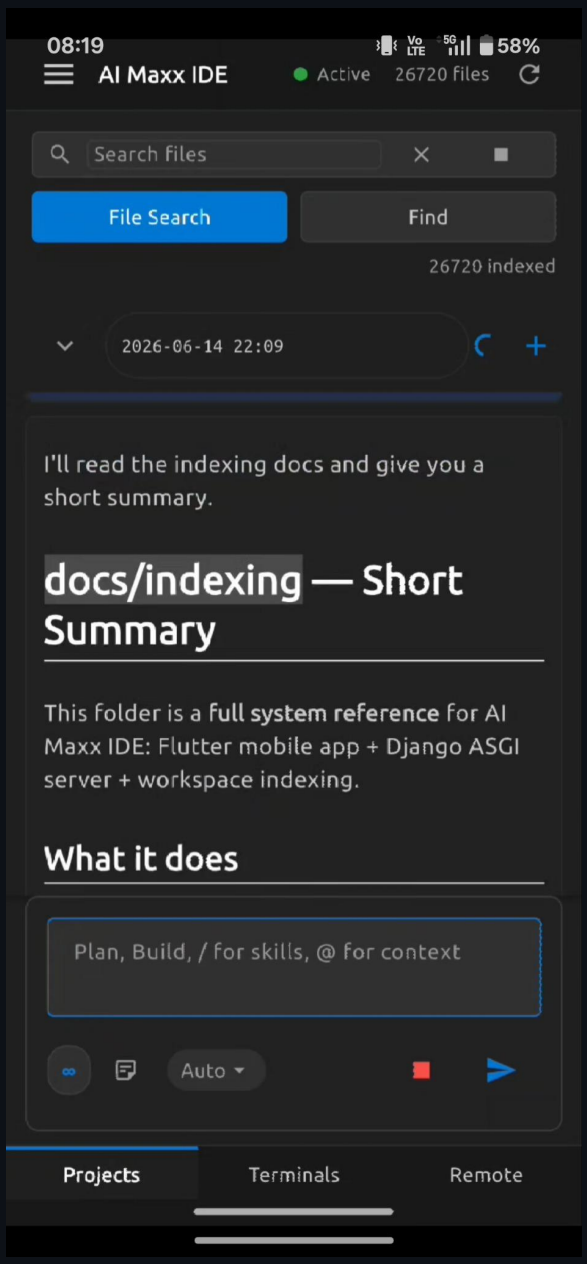
Agent Mode flips the direction. Instead of producing text, it produces actions — writing files, running commands, making changes. The same @path and / skill syntax applies, but the output drives the development environment rather than informing the developer.

The stop button visible during generation reflects that responses are streamed, not delivered in a single block. Long responses are readable as they arrive.

Plan Mode
Read and explain — analysis, summaries, architecture review

Agent Mode
Act and change — writes files, runs commands, executes tasks

Streaming
Responses arrive incrementally, readable mid-generation



Integrated Terminal

The terminal exists because some things cannot be abstracted. Git operations, build scripts, environment checks, process management — these require a real shell, not a simulated one.

Each terminal tab runs an independent subprocess with its own PID, shown in the footer. This means you can have a server running in one tab and run git commands in another simultaneously — the same mental model as a desktop development setup, just on mobile.

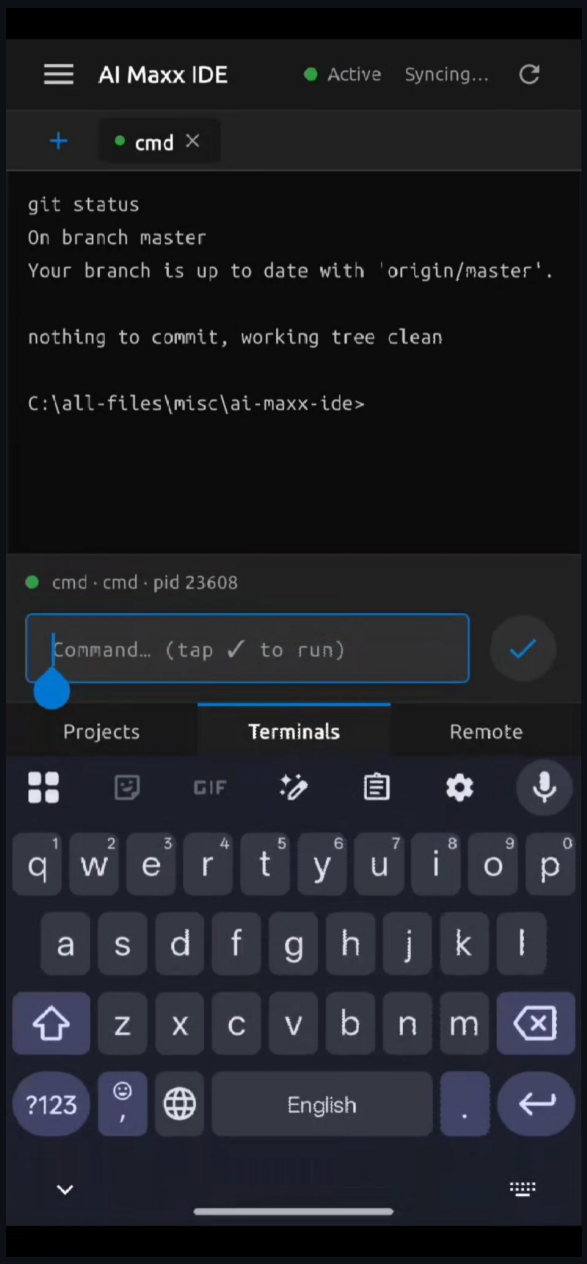
The command bar at the bottom separates input from output clearly. You type, tap ✓, and the command runs. The output scrolls above. This interaction model works better on touchscreen than trying to position a cursor inside a terminal buffer.

Git is the primary use case. Check status, view logs, stage and commit, push to remote — the full workflow without ever opening a desktop.

Isolated tabs
Each tab is an independent subprocess — parallel workflows

Input model
Command bar + tap to run — designed for touchscreen, not cursor

Primary use case
Git workflow — status, commit, push from mobile



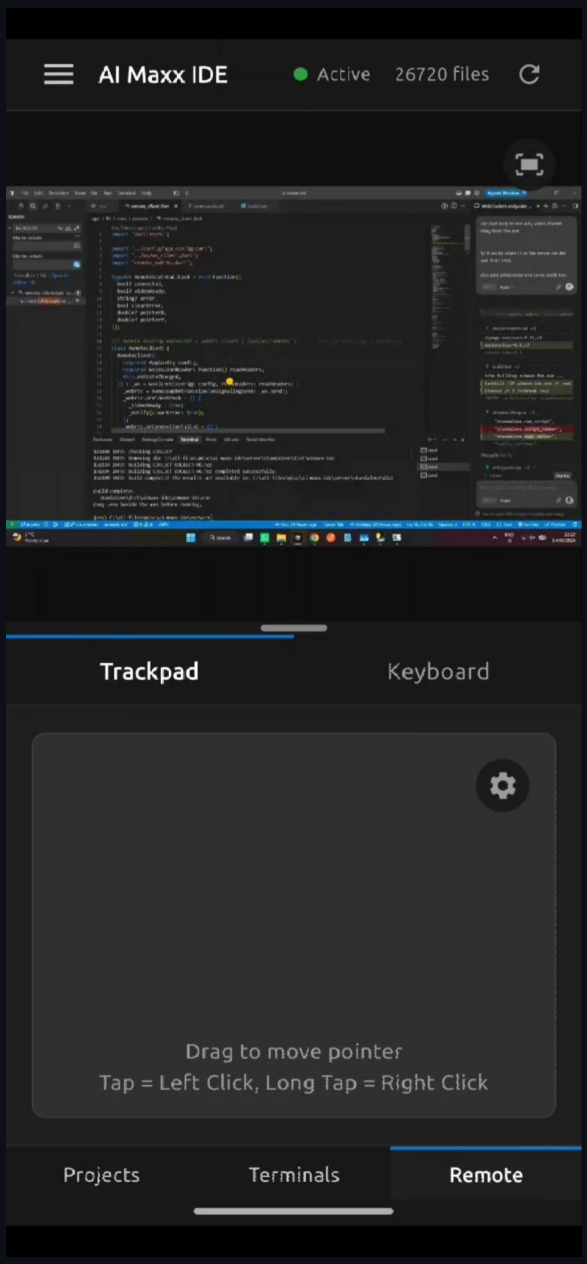
Remote Desktop — Trackpad Mode

Sometimes the abstraction breaks down entirely. A bug only reproducible in the running app, a UI that needs to be clicked through, a dialog that blocks everything else — these situations require direct control of the machine.

The Remote tab provides this as a last resort, not the primary interface. The top half of the screen shows a live stream of the host machine. Below it is a trackpad — drag to move the pointer, tap for left click, long tap for right click.

The key insight here is that the stream is read-only display; all interaction goes through the trackpad and keyboard panels rather than touch overlaid on the stream. This separation makes precision possible on a small screen — you see the result on one surface and act on another.

Cursor IDE is visible in the stream, confirming the primary target environment.



Use case

Last resort for situations that can't be handled through other panels

Stream

Read-only display — interaction is separate on trackpad/keyboard panels

Controls

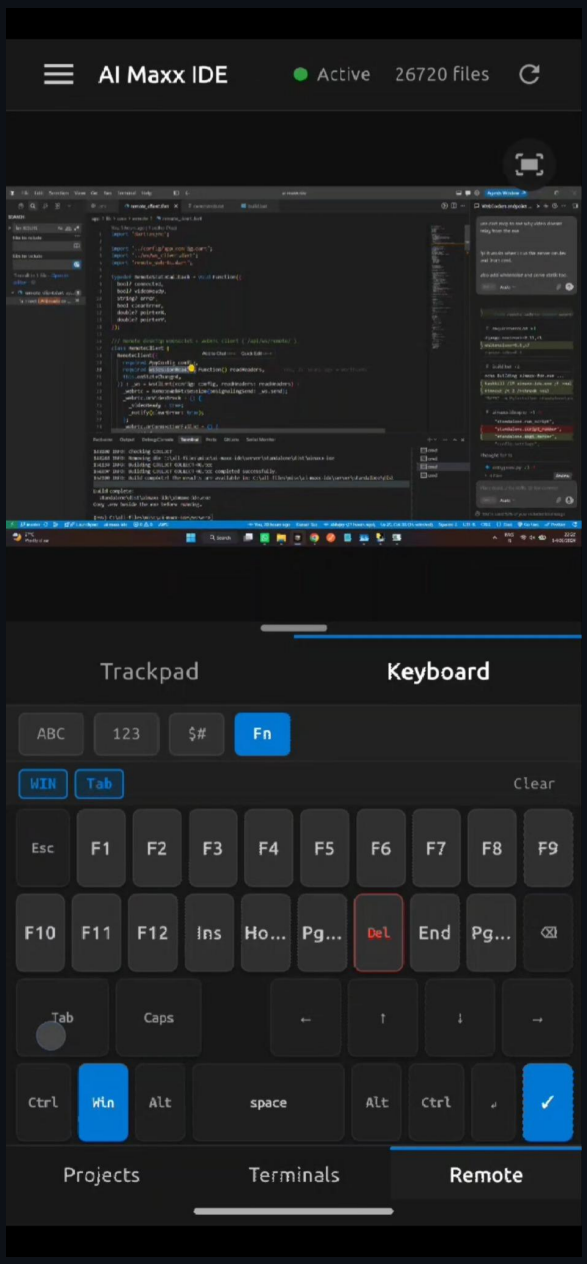
Drag = move pointer · Tap = left click · Long tap = right click

Remote Desktop — Keyboard Mode

Mouse control alone isn't enough for a developer workflow. Keyboard shortcuts are how developers move fast — switching tabs, opening command palettes, running builds, navigating editors. A developer without keyboard access to their machine is effectively locked out of their own workflow.

The virtual keyboard in AI Maxx IDE is designed specifically for this. It's not a text input keyboard — it's a shortcut keyboard. The Fn row (F1–F12), modifier keys (Ctrl, Win, Alt), Home, End, PgUp, PgDn, Del, Ins — all the keys that matter for developer shortcuts are present.

WIN and Tab are highlighted in the screenshot, suggesting an Alt+Tab or Win+Tab window switching operation. These multi-key combos are the reason this keyboard exists. Without them, remote desktop access would be limited to pointing and clicking — not real development.



Design intent
Shortcut keyboard, not text input — built for developer key combos

Key coverage
Fn row, modifiers (Ctrl/Alt/Win), navigation keys all present

Combos
Multi-key shortcuts like Ctrl+Shift+P, Alt+Tab work as expected

Operations Dashboard

Every self-hosted tool has a setup problem. The developer needs to run a server, expose it to the internet, and keep it running — before any of the actual features work. Most tools leave this as an exercise for the user.

The AI Maxx IDE dashboard makes this a two-button process. First button: run the Cloudflare Tunnel setup script. This calls cloudflared, authenticates with Cloudflare, and establishes a secure tunnel that exposes the server over the internet without opening any firewall ports or configuring NAT. The domain must be pre-configured in Cloudflare DNS, but once done, the tunnel is one click.

Second button: start the server via the batch file. The Django backend is packaged as a Windows EXE, so no Python environment, no dependency installation, no virtualenv. It runs and stays running.

Status for both tunnel and server is displayed live. If something breaks, this is the first place to look.

Tunnel
Cloudflare Tunnel — secure internet exposure, no firewall ports

Server
Django ASGI packaged as Windows EXE — no Python setup required

Dashboard
Live status for tunnel + server — single pane of operational truth